

Vulnerability Assessment Report

DownUnderNewsz - Penetration Test Report

DownUnderNewsz

22 Arthur St
Fairfield VIC 3078, Australia

September 18, 2026

Report Version: 1.0

Flinders University
Sturt Road, Bedford Park SA
5042

Table of Contents

List of Figures	3
1 Document Control	4
1.1 Team	4
1.2 List of Changes	4
2 Executive Summary	5
2.1 Overview	5
2.2 Identified Vulnerabilities	5
3 Methodology	7
3.1 Objective	7
3.2 Scope	7
3.3 User Accounts and Permissions	7
4 Findings	8
H1: Stored Cross-Site Scripting (XSS) via Article Content	8
M1: Anonymous FTP Read Access to Media Files	9
M2: Missing CSRF Protection on Administrative Endpoints	10
L1: Missing HTTP Security Headers	11
L2: Session Cookie Missing Secure and SameSite Flags	12
5 Disclaimer	13
A Appendix	14
A.1 Static Appendix Section	14
A.2 Assessment Methodology	14
A.3 Coverage and Limitations	14
A.4 Severity Rating Scale	14

List of Figures

Figure 1 - Distribution of identified vulnerabilities	6
---	---

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial release	Sep 18, 2026

2 Executive Summary

2.1 Overview

This report documents an authorized full-scope vulnerability assessment of **DownUnderNewsz**, a small independent online news outlet. The company operates a Flask-based news website with a role-based administration panel (administrator, writer, and editor roles) hosted on a virtual private server, together with a separate server used to store and serve media files.

The assessment was conducted between 17 and 18 September 2026 and covered the company's primary news website (`thedownundernews.online`) and its media host (`media.thedownundernews.online`). Five issues were confirmed in total: **one high-severity, two medium-severity, and two low-severity** findings.

Key takeaways for leadership

The most important issue is a stored cross-site scripting (XSS) vulnerability. A staff member who can publish articles (the "editor" role) could place malicious code inside an article that would then run automatically in the browser of anyone who visits the site, including other logged-in staff. This could allow an attacker to perform actions on behalf of visitors and undermine the security of the whole application. Reassuringly, the separation between staff roles, the permission to publish, and the checks on uploaded file types were all found to be working correctly.

Two medium-severity issues relate to configuration: the media server currently allows anyone on the internet to read its stored files without logging in, and the administration panel lacks protection against a type of attack known as cross-site request forgery (CSRF).

The two low-severity items concern missing security settings on website cookies and missing security headers on the web servers.

Recommended priorities

1. Sanitise article content on the server so that scripts cannot be stored inside articles.
2. Disable anonymous (unauthenticated) access to the media server's FTP service.
3. Add CSRF protection to the administration panel.
4. Apply the missing cookie flags and security headers.

Overall, the application has a sound foundation, and every issue identified can be resolved with targeted configuration and code changes. Addressing the high-severity item first will materially reduce risk to the business and its readers.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
H1	8.7	Stored Cross-Site Scripting (XSS) via Article Content	8
M1	5.3	Anonymous FTP Read Access to Media Files	9
M2	4.3	Missing CSRF Protection on Administrative Endpoints	10

#	CVSS	Description	Page
L1	3.1	Missing HTTP Security Headers	11
L2	3.1	Session Cookie Missing Secure and SameSite Flags	12

Vulnerability Overview

In the course of this penetration test **1 High**, **2 Medium** and **2 Low** vulnerabilities were identified:

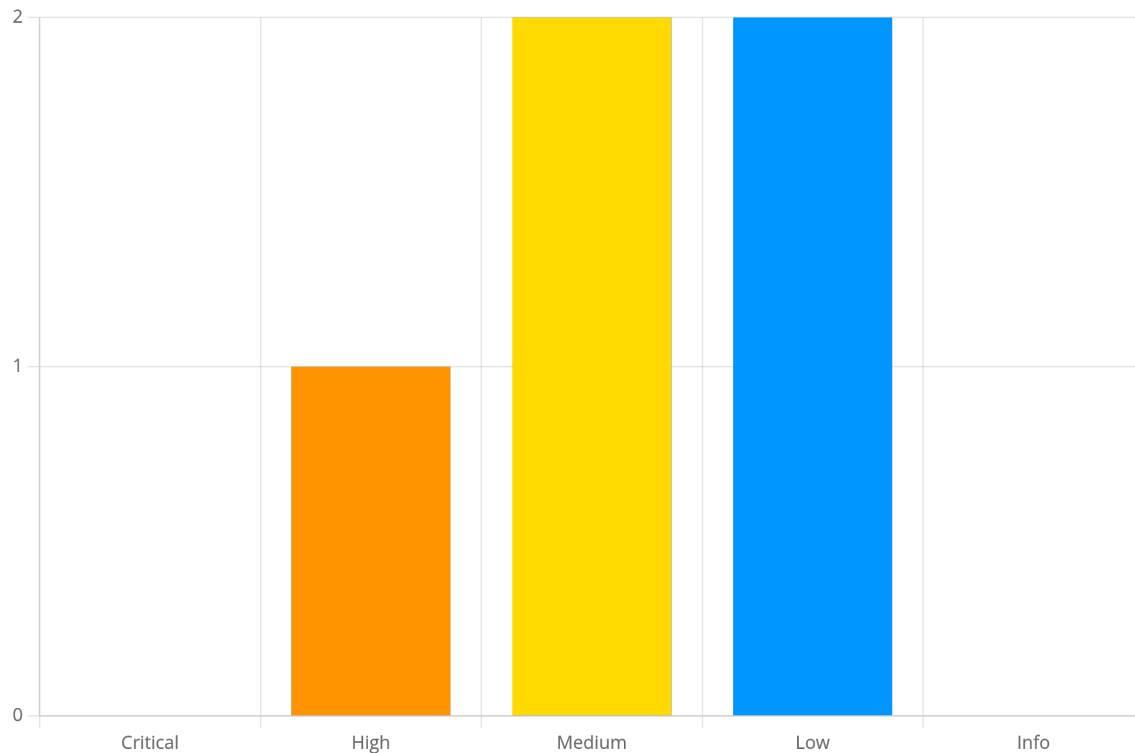


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasible, seamless and cost-affective such approach is.

3.2 Scope

The following in-scope systems were assessed during this engagement:

System	Address	Description
thedownundernews.online / www.thedownundernews.online	51.161.130. 70	Primary news web application (Flask, nginx); ports 22 (SSH) and 443 (HTTPS)
media.thedownundernews.online	51.161.131. 195	Dedicated media VPS (vsftpd 3.0.5 on 21, OpenSSH 10.0p2 on 22, nginx on 80/443)

Third-party infrastructure (OVH AS16276, Spaceship registrar/DNS/email, Google Fonts, jsDelivr, Let's Encrypt) was explicitly **out of scope**.

3.3 User Accounts and Permissions

The following owner-provided test accounts were used for authenticated testing:

- admin
- writer
- editor

4 Findings

H1: Stored Cross-Site Scripting (XSS) via Article Content	
Score	8.7 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N
Target	https://thedownundernews.online/ (article content, public + admin)
References	https://owasp.org/www-community/attacks/xss/

Overview

Any user who can publish an article (the "editor" role) can inject arbitrary JavaScript that runs in the browser of every visitor to the news site. There is no server-side sanitisation of the article body.

Details

The article "content" field is rendered as raw HTML on public article pages (rendered with a "safe"/Markup filter), while the create/update API (POST/PUT /api/admin/articles) stores the field verbatim with no sanitisation. Verified end-to-end during the assessment: a payload of `<script>alert(document.domain)</script><img src=x onerror=alert(1)>` was stored byte-for-byte, the article was published by an editor account, and the payload was returned unescaped on the public article page (executable). Writer accounts cannot publish (server enforces draft), so exploitation requires an editor or admin account. Impact is high because the stored script executes in the context of every visitor, including logged-in administrators, allowing session abuse (e.g. issuing authenticated requests) and full compromise of the application's security boundary.

Recommendation

Sanitise/allowlist HTML on the server side before storing article content (e.g. Bleach/nh3 or a strict allowlist), or render content with an HTML sanitiser. Do not rely on the client-side TinyMCE editor for security. Consider restricting the "publish" permission and applying a Content-Security-Policy.

M1: Anonymous FTP Read Access to Media Files	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	51.161.131.195 (media.thedownundernews.online) — vsftpd 3.0.5 on port 21
References	https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/04-Authentication_Testing/03-Testing_for_Weak_Authentication

Overview

The dedicated media server exposes its stored media files to anonymous (unauthenticated) FTP users. Anyone on the internet can list and read the files the CMS uploads.

Details

The media VPS (51.161.131.195, vsftpd 3.0.5) permits anonymous login. A directory listing returned media files including a PDF document: 58e75939c3d24841806d3a069e919576.png, 66ea642847be4ef0a800fb8cb8432d79.png, and d2273a1a2576489fb419cc1049c4c62f.pdf. Anonymous write (upload) was tested and correctly denied, so the exposure is read-only. Files that are intended to be internal (e.g. drafts, documents) may be exposed without authentication.

Recommendation

Disable anonymous FTP access (set anonymous_enable=NO in vsftpd.conf). Serve media only over authenticated/authorised HTTP(S) channels, and restrict access at the network level to the application server if media must remain private.

M2: Missing CSRF Protection on Administrative Endpoints

Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N
Target	https://thedownundernews.online/ (all /api/admin/* and /admin/* state-changing endpoints)
References	https://owasp.org/www-community/attacks/csrf

Overview

The admin panel performs no Cross-Site Request Forgery (CSRF) defence: no CSRF tokens, no Origin/Referer validation, and the session cookie carries no SameSite attribute.

Details

Every admin form and API call relies solely on the session cookie (access_token_cookie, a JWT) with no CSRF token. Verified during the assessment: a state-changing POST /api/admin/articles carrying a cross-origin Origin and Referer header (evil.example.com) and no CSRF token succeeded with 201 Created. The session cookie also has no SameSite attribute. Modern browsers default to SameSite=Lax, which partially mitigates cross-site POST requests, but the application itself provides no CSRF defence and this relies entirely on browser-default behaviour. Affected operations include creating/updating/deleting articles, deleting media, resetting user passwords, and deleting users.

Recommendation

Implement synchronizer-token CSRF protection on all state-changing endpoints and forms, set SameSite=Lax (or Strict) on the session cookie, and validate the Origin/Referer header for state-changing requests.

L1: Missing HTTP Security Headers	
Score	3.1 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N
Target	• https://thedownundernews.online/ • https://media.thedownundernews.online/
References	https://owasp.org/www-project-secure-headers/

Overview

The web application and media host do not set common security-related HTTP response headers.

Details

Responses from the main site contain only Server, Date, Content-Type, Content-Length and Connection. The following security headers are absent: Strict-Transport-Security (HSTS), Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy. The absence of X-Frame-Options / CSP frame-ancestors permits clickjacking of pages (including the admin panel). The absence of HSTS leaves initial connections and redirects vulnerable to downgrade. The absence of CSP reduces defence-in-depth against the stored XSS described in the first finding.

Recommendation

Add the missing headers at the nginx layer, at minimum: Strict-Transport-Security, X-Content-Type-Options: nosniff, X-Frame-Options: DENY (or CSP frame-ancestors), and a Content-Security-Policy. Set Referrer-Policy and Permissions-Policy as appropriate.

L2: Session Cookie Missing Secure and SameSite Flags

Score	3.1 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N
Target	https://thedownundernews.online/ (access_token_cookie and session cookies)
References	https://owasp.org/www-community/controls/SecureCookieAttribute

Overview

The authentication cookie is not marked Secure, and neither it nor the Flask session cookie carries a SameSite attribute.

Details

The login response sets access_token_cookie with HttpOnly and Path=/ only — no Secure and no SameSite attribute. Port 80 is open (it returns a 301 redirect to HTTPS). Because the cookie is not marked Secure, a browser would transmit the session token in cleartext if a plain-HTTP request were triggered (e.g. HTTPS stripping or a malicious link), enabling session theft by a network attacker before the redirect occurs. The missing SameSite attribute is also a factor in the CSRF finding above.

Recommendation

Set Secure and SameSite=Lax on the access_token_cookie (and session cookie). Consider enabling HSTS so browsers always use HTTPS.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerability assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section

A.2 Assessment Methodology

The assessment combined passive and enumeration reconnaissance (DNS, WHOIS/RDAP, certificate transparency, and Shodan), authenticated web-application testing using owner-provided test accounts (admin, writer, editor), targeted service/version scanning on known-open ports only, and manual verification of all findings.

A.3 Coverage and Limitations

Coverage included authentication and session management, role-based access control and IDOR, SQL injection, cross-site scripting (reflected and stored), CSRF, file-upload validation, FTP/HTTP on the media host, and HTTP security headers.

The following limitations apply: no destructive testing and no brute-force or password-spraying were performed per engagement rules (the JWT signing secret strength was not brute-forced); the media host HTTPS backend returned 502 (upstream down) during the assessment; SQL injection was tested but not exhaustively; and WHOIS registrant data is privacy-redacted.

A.4 Severity Rating Scale

Findings are rated using the CVSS v3.1 severity bands:

Rating	CVSS v3.1 Score
Critical	9.0 - 10.0
High	7.0 - 8.9
Medium	4.0 - 6.9
Low	0.1 - 3.9
Informational / None	0.0